

From Zero to First Client in 30 Days

The day-by-day guide — build the platform, teach the brain, and launch your first paying client, one day at a time

This is the same project as the Practical Guide (4 relaxed weeks), but sliced into 30 concrete days. Each day has a clear goal, ordered steps, exact commands where they help, and a "how to know it worked" test so you never wonder whether you're done. Work at your own pace — a "day" is one focused session of 30–90 minutes. Cost to you: \$0 and roughly 30 focused sessions. The rough edges described in earlier guides are already fixed in this codebase, so the instructions below work as written.

Audience — solo founder / agency owner, technical or not

Version — v1.0.0 (fixed codebase) **Date** — September 2026

Reading time — ~60 min **Total effort** — ~30 sessions × 30–90 min

WHAT'S INSIDE

The plan at a glance

Days 1–7: Foundation

Days 8–14: First client

Days 15–21: Hardening

Days 22–30: Launch

Every day: a test

Quick-reference cards

All tools are free forever · no credit card · the only investment is one focused session a day

Contents

All 32 chapters. Every entry is a link — click to jump, in the PDF or in your browser. Page numbers are the printed page numbers.

The Plan at a Glance	5
Day 1 — Create Your 6 Free Accounts	7
Day 2 — Prepare the Repo & Secrets	9
Day 3 — Deploy the Backend	11
Day 4 — Deploy Widget & Admin	13
Day 5 — Local Dev Environment	15
Day 6 — Tour the Codebase	18
Day 7 — Buffer & Your Stack Card	20
Day 8 — Create Your First Tenant	21
Day 9 — Upload Knowledge	23
Day 10 — Business Hours & Slots	25
Day 11 — Test Every Chat Path	27
Day 12 — Emails End to End	29
Day 13 — Brand the Widget	31
Day 14 — Buffer & Onboarding Checklist	32
Day 15 — Onboard a Second Tenant	33
Day 16 — Monitoring & Alerts	34
Day 17 — Security Pass	35
Day 18 — Backup & Recovery	36
Day 19 — Build Your Sales Kit	37
Day 20 — Quality Pass	38

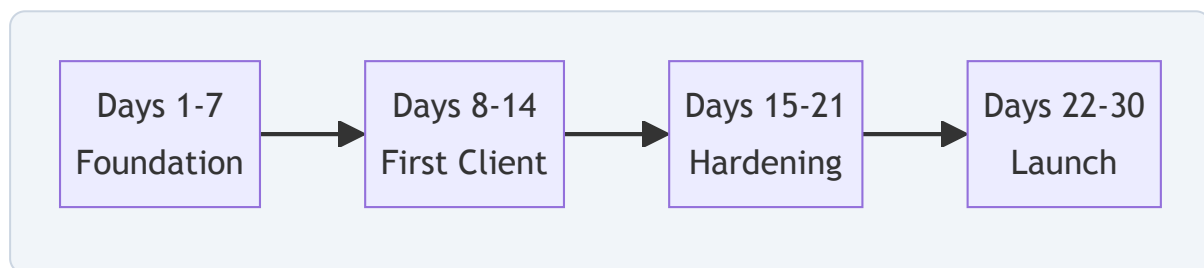
Day 21 — Buffer & Dry-Run	39
Day 22 — Landing Page & Offer	40
Day 23 — Outreach Day 1	41
Day 24 — Outreach Day 2 & Demos	42
Day 25 — Pilot Kickoff	43
Day 26 — First-Week Check-In	44
Day 27 — Case Study & Testimonial	45
Day 28 — Business Review	46
Day 29 — Scale & Automate	47
Day 30 — Celebrate & 90-Day Plan	48
Quick Reference Cards	49

⚠ **STACK UPDATE (SEPTEMBER 2026) — READ THIS FIRST.** This guide was written when the backend ran as a **DOCKER SPACE ON HUGGING FACE**. HF removed the Docker-SDK free tier in July 2026 (only Static Spaces are free), so the codebase now ships a **CLOUDFLARE WORKER** backend (`worker/`) with **GOOGLE GEMINI** for chat + embeddings. Before starting, follow `DEPLOY.md` / `SETUP.md` for the current steps and use the Worker URL `https://chatbot-api.<subdomain>.workers.dev` wherever a chapter says `...hf.space`. The admin dashboard is **MOSTLY LIVE** (Clerk sign-in, tenant list/create/detail/**DELETE**, analytics — Day 4 + Day 8 onward). Chapters about the remaining later features (appointments, emails, PDF upload in the Worker, settings PATCH) are the **ROADMAP** — they land in Phases 2–4, described in the Appendix “Migration Roadmap” at the end of this guide. The old Python backend is kept in `backend/` as the reference implementation.

The Plan at a Glance (10 min read)

Goal: see the whole month before you start, so every day has a place in the plan.

You are building a **multi-tenant AI chatbot**: one backend that powers many clients, each with their own widget, colors, knowledge base, business hours, and appointment calendar. Clients pay you monthly. This guide takes you from an empty laptop to a first paying client in 30 focused sessions.



Phase	Days	Theme	You end with
1. Foundation	1–7	Accounts, secrets, deploys, local dev	A live backend at <code>https://chatbot-api.<subdomain>.workers.dev/health</code> , a downloadable widget, a login screen, and a stack card
2. First client	8–14	Tenant, knowledge, hours, chat, email	A fully working demo client: branded widget that answers, books, and emails
3. Hardening	15–21	Second client, monitoring, security, sales kit	A repeatable < 30-minute onboarding, alerts, and a proposal you can send

4. Launch	22–30	Landing page, outreach, pilot, review	A live pilot client, a case study, and a written 90-day plan
-----------	-------	---------------------------------------	--

How each day works

- **Goal** — one sentence: what "done" looks like.
- **Steps** — ordered actions, with exact commands where a command helps.
- **How to know it worked** — a concrete test at the end of the day.

PACING. A "day" is one focused 30–90-minute session. Skipped a day? Just do the next one — nothing expires. If a day takes far longer than its estimate, stop, note what blocked you, and move on; most "stuck" moments are a typo in a slug, a missing secret, or a deploy still in progress (wait 5–10 minutes before debugging).

Day 1 — Create Your 6 Free Accounts (60 min)

Goal: every account exists, and you've saved the credentials you'll need in one place (a notes app, a spreadsheet, or a paper card). No credit card is required anywhere.

Steps

1. **GitHub** — go to `github.com`, sign up, and enable two-factor authentication (Settings → Password and authentication). You'll host the code here and GitHub Actions will do every deploy for free.
2. **Cloudflare** — sign up, then create an API token (My Profile → API Tokens → template “Edit Cloudflare Workers” + Account → Pages:Edit) and copy your Account ID. This replaces Hugging Face: the backend now runs as a Cloudflare Worker, which is free and never sleeps, and your widget + admin dashboard will be served from Cloudflare Pages under this same account. Hugging Face Docker Spaces are no longer free (July 2026).
3. **Google AI Studio** — at `aistudio.google.com/apikey` create a Gemini API key (`GEMINI_API_KEY`). This replaces the local Ollama model — chat and embeddings run on Gemini's free tier.
4. **Turso** (the database) — go to `turso.tech`, sign up, click Create Database, name it `chatbot-db`, pick the region closest to your clients. After creation, open the database → Show connection details and copy the **Database URL** (`libsql://...`) and the **Auth Token**.
5. **Clerk** (login for your admin dashboard) — go to `clerk.com`, sign up, create an application named `Chatbot Admin`. Copy the **Publishable Key** (`pk_test_...`) and the **Secret Key** (`sk_test_...`). In the Clerk dashboard: Users & Authentication → Email, Phone, Username → enable **Email** only. **Skip the Allowlist** — that is a paid (Pro) feature on the free plan; instead leave sign-up open ("Every user can sign up") or use **Invitations** to invite your own email. Sign in once to register your email, then close the window. Your email is what matters, and the backend gates admin access by the `ADMIN_EMAILS` secret regardless of who can sign up (Day 17). (You'll add your admin dashboard domain on Day 4.)

6. **Google Apps Script** (sends your emails through Gmail) — go to

`script.google.com` , New Project, delete the default code, paste the whole contents of `gas-email/Code.gs` from this repo, then Deploy → New deployment → gear icon → type: **Web app** → Execute as: **Me** → Who has access: **Anyone** → Deploy. Copy the Web App URL (it ends in `/exec`). Finally, run the `testEmail()` function once in the editor to authorize Gmail access.

HOW TO KNOW IT WORKED: you have a credentials sheet with exactly these values

— `TURSO_DATABASE_URL` , `TURSO_AUTH_TOKEN` , `GEMINI_API_KEY` ,
`CLERK_PUBLISHABLE_KEY` , `GAS_WEBAPP_URL` (Phase 2), `ADMIN_EMAILS` ,
`ADMIN_BOOTSTRAP_TOKEN` (optional), `CLOUDFLARE_API_TOKEN` , and
`CLOUDFLARE_ACCOUNT_ID` . The test email arrived in your Gmail (if you set up Apps Script).

Day 2 — Prepare the Repo & Secrets (30 min)

Goal: the code lives in your GitHub account and all the secrets are stored so the deploy workflows can use them.

Steps

1. Open this repository on GitHub and click **Use this template** → create your own repo (e.g. `chatbot-saas`) under your account.
2. Clone it to your machine:

```
git clone https://github.com/YOUR_USERNAME/chatbot-saas.git
cd chatbot-saas
```

3. Go to your repo → Settings → Secrets and variables → Actions → New repository secret, and add the values from Day 1. The exact secret names are required — copy them precisely:

```
TURSO_DATABASE_URL
TURSO_AUTH_TOKEN
GEMINI_API_KEY
CLERK_PUBLISHABLE_KEY
ADMIN_EMAILS
ADMIN_BOOTSTRAP_TOKEN    # optional – first onboarding only (DEPLOY.md §5)
CLOUDFLARE_API_TOKEN
CLOUDFLARE_ACCOUNT_ID
GAS_WEBAPP_URL           # needed in Phase 2
```

4. **Optional:** `CLERK_JWKS_URL` — only add this if your Clerk key is unusual; normally the Worker derives the token-verification URL from the publishable key automatically.

HOW TO KNOW IT WORKED: the Actions → Secrets page lists exactly 11 names with no typos. (A typo here is the #1 cause of "works locally, fails in production" — double-check the Turso URL and token especially.)

WHY SECRETS, NOT FILES? The deploy workflows (in `.github/workflows/`) read these secrets at build time and inject them into the container as environment variables. The app refuses to start without all six backend settings, so missing secrets show up immediately as a failing health check on Day 3.

Day 3 — Deploy the Backend (45 min, mostly waiting)

Goal: your AI backend is live on **Cloudflare Workers** answering the health check; the Turso database has the schema loaded; your Gemini API key is confirmed working.

Steps

1. Make sure the GitHub secrets from Day 2 are set (especially `TURSO_DATABASE_URL` , `TURSO_AUTH_TOKEN` , `GEMINI_API_KEY` , `CLOUDFLARE_API_TOKEN` , `CLOUDFLARE_ACCOUNT_ID`).
2. Push the repo to `main` (or run the workflow manually):

```
git add -A && git commit -m "Initial deploy" && git push origin main
```

This triggers `Deploy Chatbot API Worker to Cloudflare` — it typechecks `worker/` , runs `wrangler deploy` , then writes the Worker secrets. Takes 1–3 minutes. Watch it under the Actions tab.

3. When the workflow finishes, open the health check:

```
https://chatbot-api.YOUR-SUBDOMAIN.workers.dev/health
```

(`wrangler whoami` shows your subdomain.) You should see `{"status":"healthy","version":"1.0.0",...}` .

4. **Load the database schema** (the Worker talks to Turso over HTTP and needs the tables to exist):

```
turso db shell chatbot-db < worker/db/schema.sql
```

Then verify in the Turso dashboard → your database → Explore/SQL:

```
SELECT name FROM sqlite_master WHERE type='table' ORDER BY name;
```

You should see 7 tables: `appointments` , `conversations` , `end_users` , `knowledge_chunks` , `messages` , `tenants` , `usage_logs` .

5. **Confirm your Gemini key works.** No models to download — chat and embeddings run on the Gemini API. In Google AI Studio, confirm the key from Day 1 is valid and note your model's free daily quota (it varies — see the Appendix note on limits).

HOW TO KNOW IT WORKED: `/health` returns 200 with `healthy` ; all 7 tables exist in Turso; no model downloads needed.

FREE-TIER LIMITS, NOT COLD STARTS. Workers don't sleep (no keep-alive needed), but the Gemini free tier has daily request caps that vary over time. The Worker handles them gracefully — a friendly “I'm at my usage limit” message instead of an error — and you can add a Groq/Workers-AI fallback later (spec §5.7).

Day 4 — Deploy Widget & Admin (45 min)

Goal: the client-facing widget and your admin dashboard are both live on Cloudflare Pages.

Steps

1. Cloudflare dashboard → Workers & Pages → Create → Pages → Connect to Git → pick your repo.
2. Create the **widget** project: name `chatbot-widget`, production branch `main`, framework: None, build command `cd widget && npm run build`, build output directory `widget/dist` → Save and Deploy. The build emits exactly `widget.js`.
3. Create the **admin** project: name `chatbot-admin`, same branch, build command `cd admin-dashboard && npm run build`, output `admin-dashboard/dist` → Save and Deploy.
4. **Give the admin build your Clerk key.** The admin app bakes the publishable key in at build time. *Two deploy paths — pick one:*
 - **GitHub Actions (this repo's default):** the `deploy-admin.yml` workflow already bakes `VITE_CLERK_PUBLISHABLE_KEY` (from the GitHub secret `CLERK_PUBLISHABLE_KEY`) and `VITE_API_URL` (your Worker URL) into the build on every push. Just make sure the `CLERK_PUBLISHABLE_KEY` secret exists (Day 2) and push — no Pages environment variables needed.
 - **Manual Pages git integration:** in the `chatbot-admin` Pages project, Settings → Environment variables → add `VITE_CLERK_PUBLISHABLE_KEY = your pk_test_...` → Save → redeploy. Without it the page loads but the login screen never appears.
5. **Make sure the Worker lets your email in.** The Worker's admin API accepts only the email(s) in the `ADMIN_EMAILS` secret (set on Day 2 by `deploy-worker.yml`). It must be the exact email you registered with Clerk.
6. **Add the `email` claim to the Clerk session token.** Clerk's default token carries only IDs — the Worker checks your email, so without this claim every admin API call fails and the dashboard shows empty lists. Clerk dashboard → Sessions → **Customize**

session token → add `"email": "{{user.primary_email_address}}"` → Save.
(One-time setup; skip on Day 1, do it now that the dashboard exists.)

7. **Tell Clerk about the admin domain (optional in development).** Your `pk_test_` development key accepts any origin, so sign-in works without this. When you add a custom domain in production, add it under Clerk → Domains (and allowed redirect URLs).

8. Verify both URLs:

- `https://chatbot-widget.YOUR-SUBDOMAIN.pages.dev/widget.js` — downloads a JavaScript file.
- `https://chatbot-admin.YOUR-SUBDOMAIN.pages.dev` — shows the Clerk sign-in screen.

HOW TO KNOW IT WORKED: `widget.js` downloads; the admin URL shows a login screen; signing in with your registered email opens the dashboard and lists your tenants ("No tenants yet" is correct until Day 8 if you haven't created one via API). If the page loads but stays empty after a fresh sign-in, the missing `email` claim above is the usual cause — add it and sign out/in again.

Day 5 — Local Dev Environment (45 min, optional)

Goal: you can run and debug the project on your own machine. Since the backend is a Cloudflare Worker and both frontends build statically, "local" means `wrangler dev` for the API and Vite for the widget/admin — all talking to the same remote Turso database.

YOU CAN SKIP THIS DAY. Nothing here is required: every deploy already happens automatically from GitHub (Day 3–4). Come back to this page only when you want to edit code and test changes before pushing.

Steps

1. Install Node 20+ (visit `nodejs.org` if you don't have it). Python is no longer needed for local dev — the old `backend/venv/pip/uvicorn` steps were for the previous FastAPI stack.
2. **Give the Worker its secrets locally.** `wrangler dev` reads non-secret config from `worker/wrangler.toml` and secrets from a `worker/.dev.vars` file (gitignored). Create it with the same values you stored as GitHub secrets on Day 2:

```
cd worker
TURSO_DATABASE_URL=libsql://your-db.turso.io
TURSO_AUTH_TOKEN=your_token
GEMINI_API_KEY=your_key
CLERK_PUBLISHABLE_KEY=pk_test...
ADMIN_EMAILS=you@example.com
# ADMIN_BOOTSTRAP_TOKEN=... # optional, same as the GitHub secret
```

(These are real keys for your real database — never commit `.dev.vars`.)

3. **Run the API locally:**

```
npm install
npx wrangler dev
```

then open `http://localhost:8787/health` — it returns the same `{"status":"healthy",...}` payload as production.

4. Run the widget locally (Vite dev server, hot reload):

```
cd ../widget && npm install && npm run dev
```

Then open `http://localhost:5173`. To point it at your *local* Worker instead of the deployed one, add `data-api-url="http://localhost:8787"` to the widget's embed script tag (see Day 8).

5. Run the admin dashboard locally:

```
cd ../admin-dashboard && npm install && npm run dev
```

The dev server needs the same two build-time variables the workflow bakes (Day 4) — create a local `.env` in `admin-dashboard/` (or export them before `npm run dev`):

```
VITE_API_URL=http://localhost:8787
VITE_CLERK_PUBLISHABLE_KEY=pk_test_...
```

then open the printed URL (usually `http://localhost:5174`) and sign in with Clerk.

HOW TO KNOW IT WORKED: `npm install` and `npx wrangler dev` succeed; `localhost:8787/health` returns `healthy`; the widget dev server renders the button pointed at your local API; the admin dev server shows the Clerk sign-in and your tenant list.

LOCAL VS. DEPLOYED — SAME DATABASE. Local runs use the *same* Turso database as production, so changes you make locally (tenants, knowledge, chats) appear in the deployed widget immediately. If you ever want an isolated playground, create a second Turso database and point `.dev.vars` at it.

Day 6 — Tour the Codebase (45 min)

Goal: know where every concern lives, so fixing and extending later is a lookup, not a search.

Steps

1. Open the repo in your editor and walk the folder map:

Folder	What lives there
<code>worker/src/</code>	Your backend — a TypeScript Cloudflare Worker. <code>index.ts</code> is the router; <code>chat.ts</code> handles chat; <code>llm.ts</code> calls Gemini; <code>rag.ts</code> + <code>knowledge.ts</code> do retrieval and chunking; <code>clerk.ts</code> verifies admin JWTs; <code>admin.ts</code> has tenant/knowledge/analytics handlers; <code>db.ts</code> talks to Turso; <code>config.ts</code> + <code>tenants.ts</code> are config/tenant helpers; <code>vec.ts</code> is float32-vector encoding
<code>worker/db/schema.sql</code>	The whole database schema (7 tables) — load it into Turso once with <code>turso db shell chatbot-db < worker/db/schema.sql</code>
<code>worker/wrangler.toml</code>	Non-secret config: model names (<code>CHAT_MODEL</code> , <code>EMBED_MODEL</code>), RAG knobs (<code>TOP_K</code> , <code>SIMILARITY_THRESHOLD</code> , chunk sizes), chat throttle (<code>LLM_CHAT_RPM</code>)
<code>widget/src/</code>	The embeddable chat widget (React + Shadow DOM): <code>main.tsx</code> reads the script tag, <code>ChatWidget.tsx</code> , hooks (<code>useChat</code> , <code>useConfig</code> , <code>useSession</code>), components (<code>ChatButton</code> , <code>ChatPanel</code> , <code>MessageList</code> , ...), and <code>styles.css</code> (the look)

<code>admin-dashboard/src/</code>	Your control panel: <code>main.tsx</code> / <code>App.tsx</code> (Clerk + routes), <code>components/Layout.tsx</code> (header/nav shell that renders its children), <code>pages</code> (<code>Tenants</code> , <code>TenantDetail</code> , <code>Analytics</code> , <code>Settings</code>), <code>components</code> (<code>FileUploader</code> , <code>BusinessHoursEditor</code>), and <code>api.ts</code> (attaches your Clerk token to every request)
<code>gas-email/</code>	The Apps Script that turns <code>{to, subject, html}</code> into a Gmail send (Phase 2)
<code>.github/workflows/</code>	The 3 auto-deploys: <code>deploy-worker.yml</code> (Worker + secrets), <code>deploy-widget.yml</code> and <code>deploy-admin.yml</code> (Pages; the admin one bakes <code>VITE_API_URL</code> + <code>VITE_CLERK_PUBLISHABLE_KEY</code>)
<code>backend/</code> , <code>scripts/</code>	The <i>old</i> Python/FastAPI reference implementation and its CLI helpers — kept only as reference; not deployed, not what you edit

- Follow one message end-to-end in `worker/src/chat.ts` (routed from `index.ts`): the `/widget/chat/{slug}` handler loads the tenant, saves the user's message, classifies intent via Gemini (`llm.ts`), and either runs RAG search (`rag.ts`) + streaming synthesis or returns a friendly fallback — then streams the answer as SSE frames (`{type: content|done}`). The widget's `useChat.ts` reads those events.
- Note the files you'll touch most: `worker/wrangler.toml` (model names, thresholds, rate limit), `worker/src/chat.ts` (fallback messages), `widget/src/styles.css` (widget look), `admin-dashboard/src/pages/TenantDetail.tsx` (tenant page incl. the Delete button).

HOW TO KNOW IT WORKED: for each question — "where do I change the bot's name?", "where do I add an FAQ?", "where do I change the rate limit?" — you can point at the file without searching.

Day 7 — Buffer & Your Stack Card (30 min)

Goal: catch up on anything that slipped, and create the single page that runs your operation.

Steps

1. Re-visit any day that wasn't fully green. Most common fixes: a secret typo, a deploy that needed a second run, or a workflow that needed a manual trigger.
2. Re-verify the three core URLs in a browser: `chatbot-api.YOUR-SUBDOMAIN.workers.dev/health` , `chatbot-widget.YOUR-SUBDOMAIN.pages.dev/widget.js` , `chatbot-admin.YOUR-SUBDOMAIN.pages.dev` .
3. Write your **Stack Card** — one page with: each service, its URL, your login, what it's for, and its free quota. Services on the card: GitHub, Cloudflare Workers (`chatbot-api`), Cloudflare Pages (`chatbot-widget` , `chatbot-admin`), Turso (`chatbot-db`), Clerk, Google AI Studio (Gemini), Apps Script. Note there are *no models to pull and no servers to wake* — the AI runs on Gemini's API and the Worker never sleeps.

HOW TO KNOW IT WORKED: all three URLs are green, and someone (or you, in a week) could run this operation from the Stack Card alone.

Day 8 — Create Your First Tenant (30 min)

Goal: a real (demo) client exists and its branded widget loads on a test page.

Steps

1. Open the admin dashboard and sign in. Click **Add Tenant** and fill in: Business name (e.g. *Weblyft Design*), Slug (lowercase letters, numbers, hyphens — e.g. `weblyft-design`), and a Primary Color (this now maps to the widget's `primary_color` correctly).
2. Verify the backend knows the tenant:

```
curl https://chatbot-api.YOUR-SUBDOMAIN.workers.dev/widget/config/weblyft-design
```

You should get JSON with your name, colors, greeting, and bot name.

3. Create a test page: a plain HTML file on your computer (or a CodePen) containing the embed line before `</body>` :

```
<script src="https://chatbot-widget.YOUR-SUBDOMAIN.pages.dev/widget.js" data-tenant="weblyft-design" data-api-url="https://chatbot-api.YOUR-SUBDOMAIN.workers.dev"></script>
```

4. Open the test page: a floating button appears bottom-right in your client's color. Click it — the greeting message shows.

HOW TO KNOW IT WORKED: the config endpoint returns the tenant's brand data, and the floating button + greeting render on your test page.

REMOVING A TENANT. To take a client offline, open the tenant's page in the admin dashboard and click the red **DELETE TENANT** button. A warning modal lists exactly what happens before you confirm:

- The tenant's chat widget **STOPS WORKING IMMEDIATELY** — the config/chat endpoints return "Tenant not found".
- The tenant disappears from the dashboard list and analytics.
- Its knowledge base, conversations and messages become inaccessible.

Deletion is a *soft delete* (the Worker sets `plan='deleted'` ; the row stays in Turso for audit). It cannot be undone from the dashboard. To purge the row and its data permanently you must delete the child rows first (`knowledge_chunks` , `conversations` / `messages` , `appointments` , `usage_logs` , `end_users`) then the `tenants` row — the schema has no `ON DELETE CASCADE` . See the deep-dive guide's Admin chapter.

Day 9 — Upload Knowledge (45 min)

Goal: the bot answers questions from the client's own documents, not from thin air.

Steps

1. Prepare the client's information. Best sources, in order of usefulness:

- A **FAQ list** — exact questions and answers (works best, because retrieval matches question-shaped text).
- A **plain-text** version of what clients ask about: services, prices, policies, contact info, locations.
- A **PDF** — must be real text, not scanned images; and it must be extracted to text first (the Worker doesn't parse PDFs — see the warning below).

2. Admin dashboard → open the tenant → **Knowledge tab** → **FAQ (JSON)** → paste your Q&A pairs and click Upload:

```
[{"question": "What are your hours?", "answer": "Mon-Fri 9am-5pm"},  
{"question": "Do you offer refunds?", "answer": "Yes, full refund if  
cancelled 24h before"}]
```

The dashboard posts this to `/admin/tenants/<id>/knowledge/text` and shows the created-chunk count.

3. **Longer plain-text documents:** the current Worker accepts text through the same API, so upload with curl (the knowledge endpoint takes JSON, not file uploads):

```
curl -X POST "https://chatbot-api.YOUR-  
SUBDOMAIN.workers.dev/admin/tenants/TENANT_ID/knowledge/text" \  
-H "Authorization: Bearer YOUR_ADMIN_TOKEN" -H "Content-Type: \  
application/json" \  
-d '{"source_id":"services","source_type":"txt","content":"We offer web  
design and SEO. We are open Mon-Fri 9am-5pm."}'
```

Use a Clerk session token or (if still enabled) your bootstrap token as

`YOUR_ADMIN_TOKEN` . Re-uploading the same `source_id` replaces that source.

4. Confirm chunks were created: the Knowledge tab lists the source with a chunk count.

Deeper check in Turso:

```
SELECT source_id, COUNT(*) FROM knowledge_chunks
WHERE tenant_id = (SELECT id FROM tenants WHERE slug = 'weblyft-design')
GROUP BY source_id;
```

5. Test in the widget: ask a question whose answer is in the document. The bot should answer from it (it streams the answer token by token).

HOW TO KNOW IT WORKED: the source shows "N chunks", and the widget answers a question that only the uploaded document could answer.

PDFS AREN'T PARSED BY THE WORKER (FREE-PLAN CPU LIMIT). To teach the bot from a PDF, extract its text first — on your machine (e.g. `pdftotext file.pdf file.txt`) or in the browser later (Phase 3) — then upload that text with the curl step above. A scanned PDF has no text to extract at all.

IF IT SAYS "I DON'T HAVE THAT INFORMATION": the question didn't match any chunk closely enough (the similarity threshold is 0.7, set in `worker/wrangler.toml`). Rewrite the FAQ using the words customers actually use, or add the exact Q&A pair. If the bot answers but is wordy, that's the synthesis prompt — fine for now, tune later (Day 20).

Day 10 — Business Hours & Slots (30 min, Phase 2 preview)

Goal: understand where appointment data lives, so the booking feature (Phase 2) is a day's work once it lands.

HONEST STATUS: BOOKING ISN'T IN THE WORKER YET. The schema columns exist (`business_hours`, `timezone`, `slot_duration`, `buffer_minutes` on the `tenants` table) and the Settings tab's editor exists in the dashboard code, but the live Worker does **NOT** yet expose the availability/slot endpoints or the settings-save (PATCH) route — those ship in Phase 2 (see the Appendix roadmap). Right now, asking the bot for a booking time returns: *"Appointment booking is coming soon — right now I can answer questions about the knowledge base."* That is expected, not a bug.

Steps

1. Confirm the bot handles booking intents gracefully in the widget: ask *"Book Tuesday at 2pm"* — you get the friendly "coming soon" note, and the chat thread stays healthy.
2. Look at the schema so you know the shape for later (Phase 2 fills it in):

```
SELECT timezone, slot_duration, buffer_minutes, business_hours FROM tenants
WHERE slug = 'weblyft-design';
```

`business_hours` is JSON, e.g. `{"monday": {"open": "09:00", "close": "17:00"}, ...}` with empty open/close meaning closed.

3. **Optional — pre-fill hours now** so Phase 2 is plug-and-play (or set them from the dashboard Settings tab once the PATCH route lands). Via Turso SQL:

```
UPDATE tenants SET timezone = 'Asia/Kolkata', slot_duration = 30,  
buffer_minutes = 15,  
    business_hours = '{"monday":{"open":"09:00","close":"18:00"},"tuesday":  
{"open":"09:00","close":"18:00"},"wednesday":  
{"open":"09:00","close":"18:00"},"thursday":  
{"open":"09:00","close":"18:00"},"friday":  
{"open":"09:00","close":"18:00"}}'  
WHERE slug = 'weblyft-design';
```

HOW TO KNOW IT WORKED: booking questions get the friendly "coming soon" reply (never an error), and the tenant row shows the hours columns ready for Phase 2.

Day 11 — Test Every Chat Path (60 min)

Goal: prove the five intents work, from both the API and the widget UI.

Steps

1. Open the widget on your test page and send these messages, one at a time:
 - *"What are your hours?"* → answer from your knowledge base (general_query + RAG).
 - *"What times are open on Friday?"* → a list of clickable slots (check_availability).
 - *"Book Tuesday at 2pm"* → either "booked" or "that slot is taken, here are alternatives" (book_appointment).
 - *"Cancel my appointment"* → asks for the booking email (cancel_appointment).
 - *"Talk to a human"* → the handoff message (transfer_human).
2. When the slot picker appears, click a slot to book, and click the X on the picker (this used to crash — it now dismisses cleanly).
3. Verify conversation continuity: send a message, refresh the page, and confirm the thread resumes (the widget saves the conversation id to localStorage).
4. Confirm the API persists everything — get the conversation id from the widget (or the DB), then:

```
curl "https://chatbot-api.YOUR-  
SUBDOMAIN.workers.dev/widget/history/weblyft-design?  
conversation_id=REPLACE_ME"
```

5. Spot-check with curl for one path:

```
curl -N -X POST "https://chatbot-api.YOUR-  
SUBDOMAIN.workers.dev/widget/chat/weblyft-design" \  
-H "Content-Type: application/json" \  
-d '{"message": "What are your hours?", "session_id": "test-session-1"}'
```

You'll see SSE frames: `data: {"type":"content",...}` then `data: {"type":"done",...}` .

HOW TO KNOW IT WORKED: all five intents produce the expected response, the slot picker books (and closes), a refresh continues the conversation, and the history endpoint returns the messages.

Day 12 — Emails End to End (45 min)

Goal: bookings trigger real emails: a confirmation to the customer and a notification to the business.

Steps

1. Confirm the Apps Script is authorized and reachable: run `testEmail()` once more in the Apps Script editor, then from your machine:

```
python scripts/test_gas_email.py --to you@example.com
```

Check both your inbox and the Gmail Sent folder.

2. Set the business notification address. Tenant settings can carry a `notification_email`; set it in the tenant's `settings` JSON (via Turso dashboard SQL or a small script):

```
UPDATE tenants SET settings = json_set(settings, '$.notification_email',  
'you@example.com')  
WHERE slug = 'weblyft-design';
```

3. Book an appointment for yourself through the API with your real email:

```
curl -X POST "https://chatbot-api.YOUR-SUBDOMAIN.workers.dev/widget/appointments/weblyft-design" \  
-H "Content-Type: application/json" \  
-d '{"start_time": 1780000000, "end_time": 1780001800, "email":  
"you@example.com", "name": "Test User", "title": "Consultation"}'
```

(Pick `start_time` / `end_time` from a slot the availability endpoint returned.)

4. Verify: you receive the **customer confirmation**, and the business notification arrives at `notification_email`.

HOW TO KNOW IT WORKED: two emails arrive (customer confirmation + business notification) and both appear in Gmail's Sent folder.

ONE HONEST LIMITATION. The *chat* flow books appointments without collecting the visitor's email, so it only notifies the business (no customer email from chat). The API endpoint above takes an email and emails both parties. If you want customer emails from chat, capture the visitor's email in the widget (a small extension: add an email field / extract it from the message) and pass it to `send_appointment_confirmation` . Watch the 100-email/day Gmail quota.

Day 13 — Brand the Widget (30 min)

Goal: the widget looks like the client's brand on desktop and mobile.

Steps

1. In the admin dashboard → Tenant → Settings, confirm: primary color (the button + accents), greeting (the first message), bot name (the header), and optionally `logo_url` (host the client's logo somewhere public and set it — via the settings JSON or a PATCH).
2. Verify the config endpoint reflects everything, then hard-refresh the test page. The button, header, and greeting should match the brand.
3. Test on a phone viewport (DevTools mobile mode or a real phone): the panel fits, the input is usable, the slot picker scrolls.
4. Confirm continuity again: chat a little, reload — the thread and unread badge behave.

HOW TO KNOW IT WORKED: the widget matches the brand on desktop and mobile, and a reload continues the conversation.

Day 14 — Buffer & Onboarding Checklist (30 min)

Goal: your client onboarding is a written checklist, and you've timed yourself doing it.

Steps

1. Write the repeatable checklist (you'll reuse it for every client):
 1. Create tenant (name, slug, color) — dashboard.
 2. Upload knowledge (FAQ/text/PDF) — Knowledge tab.
 3. Set hours, timezone, slot duration, buffer — Settings tab.
 4. Optional: `notification_email` in settings JSON.
 5. Test chat: general query, availability, booking, cancel.
 6. Give the client the one-line embed code.
2. Re-run the whole flow for a fresh demo tenant (create one with slug `demo` or use the seed script) while timing yourself.
3. Note anything that took longer than expected and fix it now (e.g. prepare a standard FAQ template file you can copy per client).

HOW TO KNOW IT WORKED: you have a written checklist and a measured time — target under 30 minutes from tenant creation to a live, tested widget.

Day 15 — Onboard a Second Tenant (45 min)

Goal: prove the process repeats — two independent clients, isolated data, no cross-talk.

Steps

1. Onboard a second tenant end-to-end using your Day-14 checklist. Use a different color, different knowledge (write a small fake FAQ), different hours.
2. Verify **isolation**: in the widget for tenant A, ask a question that only tenant B's knowledge knows — tenant A must not answer it.
3. Verify appointments don't leak: book a slot for tenant A, then check tenant B's availability at the same time — it must still be free.
4. Time yourself. Compare with Day 14.

HOW TO KNOW IT WORKED: both tenants work independently — separate colors, separate knowledge, separate calendars — and your onboarding time is close to or under 30 minutes.

Day 16 — Monitoring & Alerts (45 min)

Goal: you get told when the site is down — before a client tells you.

Steps

1. **No keep-alive needed:** Cloudflare Workers don't sleep, so the old 30-minute keep-alive workflow is gone. Watch Cloudflare's request counters and your Gemini quota instead (dashboard → Workers → `chatbot-api` → Analytics).
2. **External uptime monitor:** set up a free monitor on `https://chatbot-api.YOUR-SUBDOMAIN.workers.dev/health` checking every 60 seconds with notifications to Telegram/email. Options: Uptime Kuma (self-hosted, free) or healthchecks.io (free tier). Test the alert by pausing/stopping the Space for a couple of minutes.
3. **Logs:** learn where to look when something breaks (Space page → Settings → Open in terminal):

```
tail -f /var/log/supervisor/fastapi_err.log # the app's errors
# Worker logs: Cloudflare dashboard → Workers → chatbot-api → Logs
tail -f /var/log/nginx/error.log           # rate-limit 503s, 404s
```

4. **Quotas:** Gmail — run `checkQuota()` in Apps Script (100/day). Turso — dashboard shows reads vs. the 1 B/month free allowance; RAG searches read the most.

HOW TO KNOW IT WORKED: an alert arrives when the Space is down, keep-alive runs are green, and you know which log to open for each symptom.

Day 17 — Security Pass (45 min)

Goal: confirm the fixes from this codebase's hardening hold up: only you can manage tenants, and nothing sensitive is publicly exposed.

Steps

1. **Admin only for your email:** with a valid Clerk token from a different account, admin calls return 403 (the Worker checks the token's `email` claim against `ADMIN_EMAILS`). Sign out of Clerk in another browser and confirm you can't read `/admin/tenants` .

```
curl -i "https://chatbot-api.YOUR-SUBDOMAIN.workers.dev/admin/tenants"
# → 401 Missing Authorization header (no token)
# → 403 Not authorized (token from an email not in ADMIN_EMAILS)
```

2. **No public model server:** `https://chatbot-api.YOUR-SUBDOMAIN.workers.dev/ollama/` returns 404 — the model lives in the Gemini API, not in your stack.
3. **Apps Script URL stays private:** the `/exec` URL is the only secret protecting your Gmail sends. Don't publish it anywhere; anyone with it can send email as you (up to 100/day).
4. **Secrets hygiene:** confirm no `.env` files are committed; rotate any key you pasted into a chat or public gist. Delete the `ADMIN_BOOTSTRAP_TOKEN` GitHub secret now that Clerk login works (the Worker falls back to Clerk-only once the secret is gone).
5. **Rate limits:** the Worker self-throttles each tenant's chat (`LLM_CHAT_RPM` , default 10 req/min) — a public bot can't exhaust your Gemini quota through one widget. Platform-wide abuse is capped by Cloudflare + Gemini's own daily limits.

HOW TO KNOW IT WORKED: no-token and wrong-email requests get 401/403, `/ollama/` is gone, no secret files are committed, and the bootstrap token secret is deleted.

Day 18 — Backup & Recovery (30 min)

Goal: you can restore everything (database, models, deploys) in under 30 minutes.

Steps

1. **Back up the database.** The simplest full backup is a dump from the Turso dashboard, or with the Turso CLI:

```
turso db shell chatbot-db ".dump" > chatbot-db-backup.sql
```

Store it somewhere off-machine (a private GitHub gist, Dropbox, or an encrypted drive). Schedule a weekly reminder to re-dump.

2. **Practice a redeploy.** Trigger the "Deploy Chatbot API Worker to Cloudflare" workflow manually (or push a commit touching `worker/`). When it finishes, confirm `/health` is green. No model re-downloads — models live in the Gemini API, not the container.
3. **Document restore:** write down — restore DB from dump, redeploy, pull models, re-verify health + one chat test.

HOW TO KNOW IT WORKED: you have a DB dump file, you successfully redeployed, and your written recovery runbook takes under 30 minutes.

Day 19 — Build Your Sales Kit (60 min)

Goal: you can pitch and close a client without improvising.

Steps

1. **Pricing.** Pick simple tiers, e.g.:

Tier	Price	Includes
Starter	\$49/mo	Chat + answers from their info
Plus	\$99/mo	+ Appointment booking + email confirmations
Premium	\$149/mo	+ Custom branding, logo, priority support

2. **One-page proposal.** A short document with: what it does (3 bullet points), a screenshot/GIF of your demo widget, pricing, and the 30-second install ("one line of code, we do it for you").
3. **Client install card.** A one-pager for the client with their embed line and what to expect in the first week.
4. **Demo video.** Record a 2-minute Loom (free) of your demo tenant: ask a question, book an appointment, show the email. Reuse it for every prospect.

HOW TO KNOW IT WORKED: you can send a complete proposal with pricing, proof (video), and install instructions in under 10 minutes.

Day 20 — Quality Pass (60 min)

Goal: your demo is polished enough to show prospects without apologizing.

Steps

1. Walk the dashboard as a stranger: create a tenant, upload a file, edit hours, view conversations/appointments — click everything, note anything broken or confusing.
2. Harden the bot's answers: add the top 10 questions every client of this type asks as FAQ entries (price, hours, location, cancellation, what to bring...). Re-test the previously weak answers.
3. Check the logs for a day's worth of noise (`fastapi_err.log`); silence anything repeated.
4. Optional tuning knobs if answers feel off:
 - Answer too wordy — lower `temperature` for synthesis in `backend/app/services/llm.py` (0.2 default).
 - Too many "I don't know" — lower `similarity_threshold` in `rag.py` (0.7 default) or improve the FAQ wording.
 - Answers too long — reduce `max_tokens` (800 default).

HOW TO KNOW IT WORKED: a friend (non-technical) can open the admin, find things, and get correct answers to the top questions without your help.

Day 21 — Buffer & Dry-Run (30 min)

Goal: prove you can onboard a brand-new client in under 30 minutes, live.

Steps

1. Create a throwaway tenant (slug like `dry-run`) and run the whole Day-14 checklist with a stopwatch running: tenant → knowledge → hours → notification email → chat tests → embed line.
2. Record where time went. Fix the top bottleneck (usually: preparing the FAQ, or re-typing settings).
3. Delete the throwaway tenant in the dashboard: open it → **Delete Tenant** → read the warning → confirm (soft delete: `plan='deleted'` hides it everywhere; see Day 8's tip for what happens and how a permanent purge differs).

HOW TO KNOW IT WORKED: full onboarding measured under 30 minutes, with the bottleneck fixed and noted.

Day 22 — Landing Page & Offer (90 min)

Goal: one link you can send prospects that sells the service — and demonstrates it.

Steps

1. Build a simple one-page site (Carrd, Notion, or a plain HTML file on Cloudflare Pages — all free). Sections: headline ("Never miss a customer again — AI chat + booking for local businesses"), 3 benefits, how it works (3 steps), pricing table, and a call-to-action.
2. Embed your **own widget** on the page using your demo tenant's slug — the page literally demonstrates the product. Set a friendly greeting like "Hi! I'm the demo bot — ask me about our services or book a slot."
3. Make "Book a demo" go through the widget's booking flow (or link to a booking slot on your calendar).
4. Write the one-line value prop you'll use in every outreach message: e.g. "I set up AI assistants that answer your customers' questions and book appointments 24/7 — first two weeks free, then \$X/month."

HOW TO KNOW IT WORKED: the page is live, the widget works on it, and you have a one-line pitch you're comfortable sending.

Day 23 — Outreach Day 1 (60 min)

Goal: 20 businesses on your list, 10 contacted, at least one conversation started.

Steps

1. Build the list in a spreadsheet: columns = business name, category, city, contact (email/Instagram/DM), status. Target categories where "missed leads while closed" hurts: dentists, hair/nail salons, plumbers/HVAC, electricians, tutors, consultants, physios, auto shops, restaurants (reservations), real estate agents.
2. Write one personalized message per business (mention something specific: their reviews, their site, their hours). Keep it short:

"Hi [Name] — I saw [Business] gets [specific observation]. I build AI chat assistants that answer your customers' questions and book appointments while you're closed — first 2 weeks free, no commitment. Want me to set up a demo on your site?"

3. Send 10 messages (email or DM). Log statuses in the sheet.

HOW TO KNOW IT WORKED: 10 messages sent, statuses logged, and at least one reply or "tell me more" in your inbox.

THE PILOT OFFER. Free 2-week pilot (your cost is ~1 hour of setup) removes the risk objection. Convert at the end: "want to keep it on?" Most will say yes at \$49–99/mo.

Day 24 — Outreach Day 2 & Demos (60 min)

Goal: the list is fully contacted, follow-ups sent, and at least one demo call booked.

Steps

1. Send the remaining 10 messages from your list.
2. Follow up with Day-23 replies (a friendly nudge after 48 h is normal).
3. Book demo calls. Prep a 5-minute demo script: (1) ask what their top questions/booking pain are, (2) show your live demo tenant answering a question like theirs, (3) show a booking + confirmation email, (4) offer the free pilot with a date to install.
4. Add a "Follow-up" date column to your sheet and schedule next touchpoints.

HOW TO KNOW IT WORKED: all 20 contacted, follow-ups sent, and at least one demo call on the calendar (or a clear "no thanks" so you can move on).

Day 25 — Pilot Kickoff (90 min)

Goal: your first pilot client is live on their real website.

Steps

1. Sign the pilot (a simple one-page agreement: free 2 weeks, then \$X/month, 30-day notice to cancel).
2. Onboard them with your checklist: tenant (their real name/color), knowledge (their real FAQ/PDF — ask for it, or draft it from their site and confirm), hours/timezone/slots, `notification_email` = their inbox.
3. Install the widget: either they paste the embed line (give them the exact snippet) or you do it if they'll share a staging page. Verify on the live page: button, colors, greeting, no console errors.
4. Book a test appointment on their site with your email, confirm both emails arrive, then cancel it in the dashboard (or via the bot).

HOW TO KNOW IT WORKED: the widget is live on the client's site, answers their real questions, books and emails correctly, and the client knows what happens next (your Day-26 check-in).

Day 26 — First-Week Check-In (45 min)

Goal: the pilot is actually working in the wild, and you've captured the client's feedback.

Steps

1. In the admin dashboard, review the pilot's conversations and appointments: which questions came in, which booked, any that failed or confused the bot.
2. Fix the top issues you see: add FAQ entries for questions the bot missed, adjust hours if slots were booked at odd times, re-word anything that read badly.
3. Call the client. Ask three questions: (1) what did customers use it for? (2) did anything embarrass you? (3) if it was perfect, what would it do next? Log the answers.
4. Confirm emails are still flowing (their notification inbox).

HOW TO KNOW IT WORKED: the bot handled real traffic, you fixed what surfaced, and the client can name one concrete win from week one.

Day 27 — Case Study & Testimonial (45 min)

Goal: proof you can reuse to close the next clients.

Steps

1. Pull numbers from the admin dashboard + Turso: conversations, appointments booked, questions answered, plus anything the client told you ("saved 6 phone calls", "booked while closed").
2. Take screenshots: the widget on their site, a conversation, a confirmation email.
3. Write a 200-word case study: who, what they installed, what happened in week one, the quote.
4. Ask the client for a one-sentence testimonial and permission to use the case study (offer a month at half price or a small credit for it).

HOW TO KNOW IT WORKED: you have a written case study with real numbers and a client-approved quote you can put on your landing page.

Day 28 — Business Review (45 min)

Goal: know your real numbers: revenue, time, and your effective hourly rate.

Steps

1. Total the time you spent this month (all sessions, including setup). Be honest — it's usually 25–35 hours.
2. Total the revenue (pilot, if it converted; otherwise projected). Compute: effective hourly rate = revenue ÷ hours. With a \$99/mo pilot and 30 setup hours, month 1 is negative — that's fine; month 2 is where it flips.
3. Decide pricing for client #2 based on what you now know (don't lower it — raise it if the pilot is happy).
4. Start the ledger: one row per client with price, hours spent, and monthly revenue. This sheet is the whole business.
5. Set your next-month target: e.g. 3 clients, \$297/mo MRR, 6 h/week.

HOW TO KNOW IT WORKED: you have real numbers in a ledger, a decision on pricing, and a written next-month target.

Day 29 — Scale & Automate (60 min)

Goal: client #3 should take half the time of client #1.

Steps

1. Turn your Day-14 checklist into a template document (with a fill-in-the-blank FAQ file you adapt per client).
2. Automate the boring bits:
 - Standard FAQ template per industry (dental, salon, trades, tutoring...) — copy, edit, upload.
 - A reusable "new client" runbook with the exact commands/URLs (use the Quick Reference chapter below).
 - A weekly 30-minute maintenance slot on your calendar (logs, quotas, one improvement).
3. Pick the roadmap items that matter next (in rough order):
 1. Email capture in the chat flow (so chat bookings email the customer).
 2. Tenant-owner login (each client manages their own hours/knowledge — sells itself).
 3. Custom domain for each client's widget.
 4. Paid GPU for faster answers when clients complain about speed.

HOW TO KNOW IT WORKED: onboarding is a copy-paste template, your week has a fixed maintenance slot, and the roadmap has an agreed next item.

Day 30 — Celebrate & 90-Day Plan (30 min)

Goal: close the month deliberately and set the next three months on paper.

Steps

1. Look back: list the three things that worked (e.g. the pilot offer, the demo video, the checklist) and the three that didn't (e.g. cold emailing, a feature you overbuilt). Keep the winners, drop the losers.
2. Write the 90-day plan:

By	Target
Day 60	3 paying clients, \$250–300 MRR, onboarding under 20 min
Day 90	6–8 clients, \$600+ MRR, or a deliberate decision to raise prices / niche down

3. Schedule the recurring habits: weekly 30-min maintenance, monthly business review, quarterly price review.
4. Celebrate — you built a real product and launched it at \$0 cost. That's the hard part done.

HOW TO KNOW IT WORKED: you have a written 90-day plan, a maintenance habit on the calendar, and you can honestly name what to repeat and what to stop.

Quick Reference Cards

Goal: one place for the commands and URLs you'll use every week.

Core URLs

What	URL
Backend health	<code>https://chatbot-api.YOUR-SUBDOMAIN.workers.dev/health</code>
Widget file	<code>https://chatbot-widget.YOUR-SUBDOMAIN.pages.dev/widget.js</code>
Admin dashboard	<code>https://chatbot-admin.YOUR-SUBDOMAIN.pages.dev</code>
Tenant config	<code>.../widget/config/{slug}</code>
Availability	<code>.../widget/availability/{slug}?start_date=...&end_date=...&timezone=...</code>
History	<code>.../widget/history/{slug}?conversation_id=...</code>

Client embed line

```
<script src="https://chatbot-widget.YOUR-SUBDOMAIN.pages.dev/widget.js" data-tenant="client-slug"></script>
```

Admin API commands (curl, with your Clerk token in **TOKEN**)

```
# List tenants
curl -H "Authorization: Bearer $TOKEN" https://chatbot-api.YOUR-SUBDOMAIN.workers.dev/admin/tenants

# Create a tenant
curl -X POST https://chatbot-api.YOUR-SUBDOMAIN.workers.dev/admin/tenants \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"name":"Business Name","slug":"business-slug","primary_color":"#3B82F6"}'

# Delete (soft delete) a tenant by its ID – the dashboard's Delete Tenant button does the same
curl -X DELETE https://chatbot-api.YOUR-SUBDOMAIN.workers.dev/admin/tenants/TENANT_ID \
  -H "Authorization: Bearer $TOKEN"
```

Worker logs (Cloudflare dashboard → Workers → chatbot-api → Logs)

```
# No model downloads – chat + embeddings run on the Gemini API (worker/src/llm.ts)
# Real-time and past logs live in the Cloudflare dashboard; errors surface in the chat as friendly fallback messages.
```

Free quotas that matter

Resource	Free limit	Watch for
Gmail via Apps Script	100 emails/day	15+ emails/day is healthy at pilot scale
Turso	9 GB, 1 B reads/month	RAG searches read the most; check the dashboard graph
Cloudflare Worker	100k req/day free (10 ms CPU per request)	No sleeping; watch Gemini daily caps

Clerk	50k monthly active users	Only your admin signs in — negligible; free sessions expire every ~7 days, so expect a weekly re-login
Cloudflare Pages	Unlimited bandwidth	—

Weekly maintenance (30 min)

1. Health URL green.
2. Worker logs (Cloudflare dashboard) — new errors? Watch Gemini daily caps in Google AI Studio.
3. Gmail quota check (`checkQuota()`); Turso reads graph.
4. Review each client's conversations/appointments; answer anything the bot missed by adding FAQ entries.

THAT'S THE MONTH. From here, the business runs on the weekly habit and the 90-day plan — the platform itself needs almost no attention.

Appendix — Migration Roadmap: Phases 1–4 in Detail

Why this appendix exists: the original codebase deployed its backend as a Docker Space on Hugging Face. In July 2026 Hugging Face moved Docker (and Gradio) Spaces behind a paid PRO subscription, leaving only Static Spaces free. The project was re-planned around a **\$0 Cloudflare stack** and is being ported in phases (tracked in `hf-docker-exit-spec.md`). This guide describes the Phase-1 build; the phases below show what ships next.

Phase	What ships	Where (code)	Done when
1 — Chat + RAG  <i>implemented</i>	Widget chat (SSE), RAG answers from per-tenant knowledge, conversation history, admin-lite tenant + knowledge ingest, GitHub Actions deploys	<code>worker/</code> (<code>config</code> , <code>llm</code> , <code>rag</code> , <code>chat</code> , <code>clerk</code> , <code>admin</code>)	Widget on a test page answers from knowledge; <code>/health</code> green
2 — Appointments + email	Availability slots, booking/cancel in the chat, business-hours parsing, Google Apps Script email confirmations, <code>email_pending</code> retry UX on Gmail quota	<code>worker/src</code> ports of <code>appointments.py</code> + <code>email.py</code> ; <code>gas-email/</code>	"Book Tuesday 2pm" books + emails both sides

3 — Full admin dashboard  <i>mostly shipped</i>	Tenant CRUD incl. delete (soft delete + warning modal), analytics (GET /admin/analytics), conversations, appointments, usage, PDF upload (browser-side pdfjs-dist extraction), Clerk-only auth (bootstrap token removed)	Admin APIs in worker/src/admin.ts ; admin-dashboard/ wired via VITE_API_URL	Admin UI matches the old README feature list
4 — Cleanup & docs	Remaining docs/guides rewritten, legacy scripts/ ported to Worker API calls, old FastAPI backend/ removed once parity is proven	Repo root	Repo has no Hugging Face / Docker remnants

Key decisions locked in the spec

- LLM:** Gemini free tier (flash-class chat model set via `CHAT_MODEL` ; `gemini-embedding-001` @ 768 dims, matching the float32 BLOBs in `knowledge_chunks`). Groq / Workers-AI are optional failovers. Free-tier limits are volatile — verify in AI Studio and treat as config.
- Database:** Turso stays. Embeddings live in `knowledge_chunks` as float32 BLOBs and are ranked inside the Worker (Turso doesn't ship sqlite-vec). Schema: `worker/db/schema.sql` .
- PDFs:** not parsed in the Worker (free-plan CPU limit) — extracted in the browser or CLI, then uploaded as text (§5.6 of the spec).
- Auth:** Clerk JWKS for `/admin/*` , gated by the `email` claim against `ADMIN_EMAILS` (free-tier Clerk: no allowlist — invite or open sign-up; see Day 1). Optional `ADMIN_BOOTSTRAP_TOKEN` only for initial onboarding, deleted once Clerk login works (Day 17).
- Deploys:** GitHub Actions → `deploy-worker.yml` (Worker) + Pages workflows (widget/admin).

Reading the rest of this guide

Days 1–2 and 4–7 (accounts, secrets, static deploys, local dev) are current. Days about the backend internals (`backend/`) describe the reference implementation that the Worker is ported from. Days 10–12+ (appointments, emails, admin uploads, PDFs) are the Phase 2–4 roadmap — the endpoints exist in the old backend and are being ported in order. Where a command says `https://...hf.space/...` , substitute `https://chatbot-api.<subdomain>.workers.dev/...` . The live build steps always live in `DEPLOY.md` .

Practical Guide — Multi-Tenant AI Chatbot SaaS project | All tools free, no credit card required